AKADEMIA NAUK STOSOWANYCH
W NOWYM SĄCZU

Wydział Nauk Inżynieryjnych
Katedra Informatyki

DOKUMENTACJA PROJEKTOWA

ZAAWANSOWANE PROGRAMOWANIE

Algorytm listy dwukierunkowej z zastosowaniem GitHub

Autor:
Damian Kiełbasa

Prowadzący:
mgr inż. Dawid Kotlarski

Nowy Sącz 2026

Spis treści

1. Ogólne określenie wymagań **4**

1.1. Cel projektu 4
1.2. Wymagania funkcjonalne 4
1.3. Wymagania dotyczące dokumentacji i kontroli wersji 4

2. Analiza problemu **5**

2.1. Lista dwukierunkowa i jej zastosowania 5
2.2. Sposób działania 6
2.3. Wzorzec projektowy Factory 6
2.4. Wzorzec projektowy Iterator 6
2.5. Git i GitHub 7
2.6. Doxygen 8

3. Projektowanie **9**

3.1. Narzędzia i środowisko 9
3.2. Struktura projektu 9
3.3. Sposób używania systemu Git 9
3.4. Doxygen – konfiguracja i komentarze 10

4. Implementacja **12**

4.1. Klasa Node 12
4.2. Klasa NodeFactory 12
4.3. Klasa ListIterator 12
4.4. Klasa List 13
4.4.1. Dodawanie na początek i na koniec 13
4.4.2. Dodawanie pod indeks 14
4.4.3. Usuwanie 15
4.4.4. Usuwanie spod indeksu 15
4.4.5. Wyświetlanie 16
4.5. Program testowy i wyniki 16

5. Wnioski **19**

Literatura **20**

Spis rysunków **21**

Spis tabel **22**

Spis listingów **23**

1. Ogólne określenie wymagań

1.1. Cel projektu

Celem projektu jest napisanie w języku C++ programu realizującego **listę dwukierunkową** alokowaną na stercie. Implementacja ma korzystać ze wzorców projektowych **Factory** (fabryka) oraz **Iterator**[1], a każda klasa ma znajdować się w osobnym pliku. Poprawność działania poszczególnych metod listy ma zostać sprawdzona w funkcji `main`.

Drugim celem jest zapoznanie się z narzędziami wspierającymi pracę nad projektem: systemem kontroli wersji **Git** wraz z serwisem **GitHub** oraz generatorem dokumentacji **Doxygen**. Niniejszy dokument opisuje zarówno sam program listy dwukierunkowej, jak i sposób użycia tych narzędzi.

Przewidywane rezultaty projektu to:

- działający program z kompletem metod listy dwukierunkowej,
- dokumentacja wygenerowana automatycznie programem Doxygen (plik PDF),
- niniejsza dokumentacja projektowa przygotowana w LaTeXu,
- repozytorium na GitHubie z historią zmian pokazującą kolejne etapy pracy.

1.2. Wymagania funkcjonalne

Lista dwukierunkowa ma udostępniać operacje wymienione w tabeli 1.1. W tabeli podano również nazwy metod, którymi zostały zrealizowane poszczególne wymagania.

1.3. Wymagania dotyczące dokumentacji i kontroli wersji

Podczas realizacji projektu należało:

- wygenerować dokumentację automatycznie programem Doxygen do formatu LaTeX i z niej plik PDF,
- dołączyć do projektu dokumentację projektową w LaTeXu (niniejszy dokument),
- założyć konto na GitHubie i używać go podczas pisania programu.

[1] Opis wzorców projektowych: [2].

Tab. 1.1. Wymagania funkcjonalne i ich realizacja

Lp.	Wymaganie	Metoda
1	Dodaj element na początek listy	`addFront`
2	Dodaj element na koniec listy	`addBack`
3	Dodaj element pod wskazany indeks	`addAtIndex`
4	Usuń element z początku listy	`removeFront`
5	Usuń element z końca listy	`removeBack`
6	Usuń element z pod wskazanego indeksu	`removeAtIndex`
7	Wyświetl całą listę	`display`
8	Wyświetl listę w odwrotnej kolejności	`displayReverse`
9	Wyświetl następny element	`ListIterator::next`, `displayCurrent`
10	Wyświetl poprzedni element	`ListIterator::prev`, `displayCurrent`
11	Wyczyść całą listę	`clear`

2. Analiza problemu

2.1. Lista dwukierunkowa i jej zastosowania

Lista dwukierunkowa jest dynamiczną strukturą danych złożoną z węzłów. Każdy węzeł przechowuje wartość oraz dwa wskaźniki: na węzeł poprzedni i na węzeł następny[2]. Dzięki temu po liście można poruszać się w obu kierunkach. W odróżnieniu od tablicy elementy nie leżą obok siebie w pamięci, a ich liczba nie musi być znana z góry: każdy węzeł jest tworzony na stercie dopiero wtedy, gdy jest potrzebny.

Listy dwukierunkowe stosuje się między innymi w:

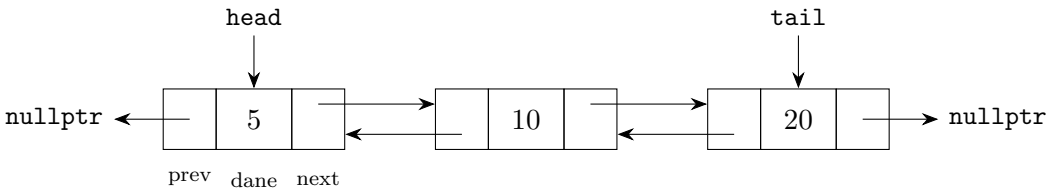
- historii przeglądarki internetowej (przejście do strony poprzedniej i następnej),
- mechanizmach cofnij/ponów (*undo/redo*) w edytorach,

[2]Podstawy struktur danych: [1].

- odtwarzaczach muzyki (utwór poprzedni i następny na liście odtwarzania),
- pamięciach podręcznych typu LRU (w połączeniu z tablicą mieszającą),
- implementacjach innych struktur, np. kolejek dwustronnych.

2.2. Sposób działania

Lista przechowuje dwa wskaźniki: `head` na pierwszy węzeł oraz `tail` na ostatni. Pole `prev` pierwszego węzła i pole `next` ostatniego mają wartość `nullptr`, co oznacza brak sąsiada. Pusta lista to taka, w której `head` i `tail` są równe `nullptr`. Strukturę listy trzyelementowej przedstawia rysunek 2.1.



Rys. 2.1. Struktura listy dwukierunkowej z trzema elementami

Złożoność obliczeniowa operacji zależy od tego, czy dostęp do elementu odbywa się przez `head` lub `tail`, czy wymaga przejścia przez kolejne węzły (tabela 2.1). Posiadanie wskaźnika `tail` i pola `prev` sprawia, że dodawanie i usuwanie na obu końcach listy jest wykonywane w stałym czasie. W liście jednokierunkowej usunięcie ostatniego elementu wymagałoby przejścia przez całą listę.

2.3. Wzorzec projektowy Factory

Wzorzec fabryki odseparowuje kod tworzący obiekty od kodu, który z nich korzysta. Zamiast wywoływać operator `new Node(...)` w wielu miejscach, lista prosi fabrykę o utworzenie węzła. Dzięki temu sposób tworzenia węzłów jest zapisany w jednym miejscu i w razie potrzeby (np. zmiany typu węzła lub liczenia utworzonych obiektów) wystarczy zmienić tylko fabrykę. W projekcie rolę fabryki pełni klasa `NodeFactory` z metodą statyczną `createNode`.

2.4. Wzorzec projektowy Iterator

Iterator udostępnia sposób sekwencyjnego przeglądania elementów kolekcji bez ujawniania jej wewnętrznej budowy. Obiekt iteratora zapamiętuje bieżącą pozycję i

Tab. 2.1. Złożoność czasowa operacji na liście dwukierunkowej

Operacja	**Złożoność**	**Uzasadnienie**
`addFront`, `addBack`	$O(1)$	dostęp przez `head`, `tail`
`removeFront`, `removeBack`	$O(1)$	dostęp przez `head`, `tail`
`addAtIndex`, `removeAtIndex`	$O(n)$	przejście do wskazanej pozycji
`display`, `displayReverse`	$O(n)$	odwiedzenie każdego węzła
`clear`	$O(n)$	usunięcie każdego węzła
`next`, `prev` (iterator)	$O(1)$	przejście do sąsiedniego węzła

potrafi przesunąć się do elementu następnego lub poprzedniego. Kod korzystający z iteratora nie musi znać szczegółów budowy węzłów ani listy. W projekcie tę rolę pełni klasa `ListIterator`, tworzona przez metodę `getIterator` listy.

2.5. Git i GitHub

Git to rozproszony system kontroli wersji[3]. Zapamiętuje kolejne stany projektu w postaci **commitów**, czyli migawek wszystkich plików wraz z opisem zmian, autorem i datą. Każdy commit można później porównać z innym, a projekt w razie potrzeby przywrócić do wcześniejszego stanu. Główne pojęcia to:

- **repozytorium** – katalog projektu wraz z historią zmian,
- **commit** – zapisany w historii stan projektu,
- **gałąź** (*branch*) – równoległa linia rozwoju projektu,
- **repozytorium zdalne** (*remote*) – kopia repozytorium na serwerze,
- **push** i **pull** – wysłanie zmian na serwer i pobranie zmian z serwera.

GitHub[4] jest serwisem internetowym hostującym repozytoria Git. Pełni rolę wspólnego, zdalnego repozytorium, dzięki któremu możliwa jest synchronizacja pracy z kilku komputerów, a także tworzenie kopii zapasowej projektu.

[3]Opis systemu Git: [5].
[4]Dokumentacja serwisu: [6].

2.6. Doxygen

Doxygen[5] to generator dokumentacji, który analizuje kod źródłowy i specjalnie sformatowane komentarze (np. `@brief`, `@param`), a następnie tworzy dokumentację w formacie HTML, LaTeX i innych. Dokumentacja ma postać opisu klas, ich pól i metod oraz diagramów zależności, dzięki czemu zawsze odpowiada aktualnemu kodowi.

[5]Podręcznik narzędzia: [7].

3. Projektowanie

3.1. Narzędzia i środowisko

Program napisano w języku C++[6]. Wymagany jest kompilator obsługujący standard C++11 (w kodzie używane jest słowo kluczowe `nullptr`). Do działania programu wystarcza biblioteka standardowa, a konkretnie nagłówek `<iostream>`; nie są używane żadne dodatkowe biblioteki.

Narzędzia wykorzystane w projekcie:

- **Visual Studio 2022**
- **Git** i **GitHub** – kontrola wersji i zdalne repozytorium,
- **Doxygen** – generowanie dokumentacji z komentarzy w kodzie,
- **Overleaf**

3.2. Struktura projektu

Zgodnie z wymaganiem każda klasa znajduje się w osobnym pliku (tabela 3.1).

Tab. 3.1. Pliki projektu

Plik	**Zawartość**
`Node.h`	klasa `Node` – węzeł listy
`NodeFactory.h`, `.cpp`	klasa `NodeFactory` – fabryka węzłów
`ListIterator.h`, `.cpp`	klasa `ListIterator` – iterator
`List.h`, `List.cpp`	klasa `List` – lista dwukierunkowa
`main.cpp`	funkcja `main` z testami metod listy
`Doxyfile`	konfiguracja programu Doxygen
`.gitignore`	lista plików pomijanych przez Git

3.3. Sposób używania systemu Git

Praca nad projektem przebiega w cyklu: zmiana kodu, sprawdzenie, czy program się kompiluje, dodanie zmian do poczekalni (`git add`) i zapisanie ich w historii

[6]Opis języka: [3], [4].

(`git commit`). Co pewien czas zmiany są wysyłane do GitHuba (`git push`). Commit powinien obejmować jedną zakończoną, niewielką zmianę (np. dodanie jednej metody), a jego opis powinien mówić, co zostało zmienione. Dzięki temu historia projektu pokazuje kolejne etapy pracy.

Podstawowe polecenia potrzebne do pierwszej konfiguracji i pracy z repozytorium przedstawia listing 1.

```
git init                                # utworzenie repozytorium
git add .                               # dodanie plikow do poczekalni
git commit -m "Dodano klase Node"       # zapis zmian w historii
git remote add origin <adres_repo> # podlaczenie repozytorium
git push -u origin main                 # wyslanie zmian na serwer
git status                              # stan plikow
git log --oneline                       # skrocona historia commitow
```

Listing 1. Podstawowe polecenia Git

3.4. Doxygen – konfiguracja i komentarze

Plik konfiguracyjny tworzy się poleceniem `doxygen -g Doxyfile`, a następnie ustawia w nim opcje z tabeli 3.2.

Tab. 3.2. Wybrane ustawienia pliku `Doxyfile`

Opcja	Wartość	Znaczenie
`PROJECT_NAME`	Lista dwukierunkowa	nazwa w dokumentacji
`OUTPUT_LANGUAGE`	Polish	język komunikatów
`EXTRACT_ALL`	YES	dokumentuj wszystko
`GENERATE_LATEX`	YES	wyjście w LaTeXu
`GENERATE_HTML`	YES	wyjście w HTML

Dokumentację generuje się poleceniem `doxygen Doxyfile`. W katalogu `latex` uruchamia się następnie polecenie `make` (w systemie Windows `make.bat`), które tworzy plik `refman.pdf`. Przykładowy komentarz w formacie rozpoznawanym przez Doxygen przedstawia listing 2.

```
/**
 * @brief Dodaje element na poczatek listy.
```

```
3  * @param val Wartosc nowego elementu.
4  */
5 void addFront(int val);
```

Listing 2. Przykład komentarza w formacie Doxygen

4. Implementacja

4.1. Klasa Node

Klasa `Node` (listing 3) reprezentuje węzeł listy. Pola są publiczne, ponieważ węzłami zarządza wyłącznie lista. Konstruktor ustawia wartość i zeruje oba wskaźniki.

```
class Node {
public:
    int data;
    Node* next;
    Node* prev;

    Node(int val) {
        data = val;
        next = nullptr;
        prev = nullptr;
    }
};
```

Listing 3. Klasa Node

4.2. Klasa NodeFactory

Fabryka (listing 4) udostępnia jedną metodę statyczną, która tworzy węzeł na stercie i zwraca wskaźnik na niego. Dzięki metodzie statycznej nie trzeba tworzyć obiektu fabryki, aby z niej skorzystać.

```
class NodeFactory {
public:
    static Node* createNode(int val) {
        return new Node(val);
    }
};
```

Listing 4. Klasa NodeFactory

4.3. Klasa ListIterator

Iterator (listing 5) zapamiętuje wskaźnik na bieżący węzeł. Metody `next` i `prev` przesuwają go do sąsiada, o ile bieżący węzeł istnieje, a `displayCurrent` wyświetla wartość bieżącego elementu albo komunikat o jego braku. Po przejściu za ostatni element iterator przyjmuje wartość `nullptr`.

```
class ListIterator {
private:
    Node* current;
public:
    ListIterator(Node* startNode) {
        current = startNode;
    }

    void next() {
        if (current) current = current->next;
    }

    void prev() {
        if (current) current = current->prev;
    }

    void displayCurrent() {
        if (current) cout << current->data << endl;
        else cout << "Brak elementu (koniec listy)." << endl;
    }
};
```

Listing 5. Klasa ListIterator

4.4. Klasa List

Klasa `List` przechowuje wskaźniki `head` i `tail`. Destruktor wywołuje metodę `clear`, więc pamięć zajmowana przez węzły jest zwalniana automatycznie przy zniszczeniu obiektu listy. Poniżej omówiono najważniejsze metody.

4.4.1. Dodawanie na początek i na koniec

Metody `addFront` i `addBack` (listing 6) tworzą węzeł za pomocą fabryki. Jeśli lista jest pusta, nowy węzeł staje się jednocześnie początkiem i końcem. W przeciwnym razie jest podłączany do dotychczasowego początku (odpowiednio końca) i aktualizowany jest wskaźnik `head` (odpowiednio `tail`).

```
void addFront(int val) {
    Node* newNode = NodeFactory::createNode(val);
    if (head == nullptr) {
        head = tail = newNode;
    } else {
        newNode->next = head;
        head->prev = newNode;
```

```
        head = newNode;
    }
}

void addBack(int val) {
    Node* newNode = NodeFactory::createNode(val);
    if (tail == nullptr) {
        head = tail = newNode;
    } else {
        tail->next = newNode;
        newNode->prev = tail;
        tail = newNode;
    }
}
```

Listing 6. Metody addFront i addBack

4.4.2. Dodawanie pod indeks

Metoda `addAtIndex` (listing 7) obsługuje trzy przypadki: indeks nie większy od zera powoduje dodanie na początek, indeks wskazujący poza koniec listy – dodanie na koniec, a każdy inny – wstawienie nowego węzła przed węzłem o podanym indeksie. Pętla zatrzymuje się, gdy osiągnie żądany indeks lub koniec listy.

```
void addAtIndex(int index, int val) {
    if (index <= 0) {
        addFront(val);
        return;
    }

    Node* current = head;
    for (int i = 0; i < index && current != nullptr; ++i) {
        current = current->next;
    }

    if (current == nullptr) {
        addBack(val);
    } else {
        Node* newNode = NodeFactory::createNode(val);
        newNode->prev = current->prev;
        newNode->next = current;
        if (current->prev != nullptr) {
            current->prev->next = newNode;
        } else {
            head = newNode;
```

```
22          }
23          current->prev = newNode;
24      }
25  }
```

Listing 7. Metoda addAtIndex

4.4.3. Usuwanie

Metody `removeFront` i `removeBack` (listing 8) zapamiętują usuwany węzeł w zmiennej pomocniczej, przesuwają wskaźnik `head` (`tail`) i dopiero wtedy zwalniają pamięć. Gdy po usunięciu lista staje się pusta, drugi ze wskaźników również jest zerowany. Metoda `clear` wielokrotnie usuwa pierwszy element, aż lista będzie pusta.

```
1  void removeFront() {
2      if (head == nullptr) return;
3      Node* temp = head;
4      head = head->next;
5      if (head != nullptr) head->prev = nullptr;
6      else tail = nullptr;
7      delete temp;
8  }
9
10 void removeBack() {
11     if (tail == nullptr) return;
12     Node* temp = tail;
13     tail = tail->prev;
14     if (tail != nullptr) tail->next = nullptr;
15     else head = nullptr;
16     delete temp;
17 }
18
19 void clear() {
20     while (head != nullptr) {
21         removeFront();
22     }
23 }
```

Listing 8. Metody removeFront, removeBack i clear

4.4.4. Usuwanie spod indeksu

Metoda `removeAtIndex` (listing 9) przekazuje usuwanie pierwszego i ostatniego elementu do metod `removeFront` i `removeBack`. Dla elementu ze środka listy łączy ze

sobą jego sąsiadów i zwalnia węzeł. Jeśli indeks wykracza poza listę, metoda nie robi nic.

```
void removeAtIndex(int index) {
    if (head == nullptr) return;
    if (index == 0) {
        removeFront();
        return;
    }

    Node* current = head;
    for (int i = 0; i < index && current != nullptr; ++i) {
        current = current->next;
    }

    if (current != nullptr) {
        if (current == tail) {
            removeBack();
        } else {
            current->prev->next = current->next;
            current->next->prev = current->prev;
            delete current;
        }
    }
}
```

Listing 9. Metoda removeAtIndex

4.4.5. Wyświetlanie

Metoda `display` przechodzi listę od `head` wskaźnikami `next`, a `displayReverse` od `tail` wskaźnikami `prev`. Dla pustej listy `display` wypisuje stosowny komunikat. Metoda `getIterator` zwraca nowy iterator ustawiony na początku listy.

4.5. Program testowy i wyniki

Działanie klas sprawdzono w funkcji `main` (listing 10). Test obejmuje dodawanie (na koniec, na początek i pod indeks), usuwanie spod indeksu, wyświetlanie w obu kierunkach, ruch iteratora do przodu i do tyłu oraz czyszczenie listy.

```
int main() {
    List list;

    // Testowanie dodawania
```

```
    list.addBack(10);
    list.addBack(20);
    list.addFront(5);
    list.addAtIndex(1, 15);
    list.display(); // Oczekiwane: 5 15 10 20

    // Testowanie usuwania
    list.removeAtIndex(1);
    list.display(); // Oczekiwane: 5 10 20

    // Testowanie wyświetlania od tyłu
    list.displayReverse(); // Oczekiwane: 20 10 5

    // Testowanie wzorca Iterator
    cout << "--- Test Iteratora ---" << endl;
    ListIterator it = list.getIterator();

    cout << "Iterator start: ";
    it.displayCurrent(); // 5

    it.next();
    cout << "Iterator next: ";
    it.displayCurrent(); // 10

    it.prev();
    cout << "Iterator prev: ";
    it.displayCurrent(); // 5

    // Czyszczenie
    list.clear();

    return 0;
}
```

Listing 10. Funkcja main z testami

Wynik uruchomienia programu przedstawia listing 11. Odpowiada on wartościom oczekiwanym zapisanym w komentarzach w kodzie testu oraz przykładowi ręcznemu z tabeli **??**.

```
Zawartosc listy: 5 15 10 20
Zawartosc listy: 5 10 20
Od tylu: 20 10 5
--- Test Iteratora ---
Iterator start: 5
Iterator next: 10
```

```
Iterator prev: 5
```

Listing 11. Wynik działania programu

5. Wnioski

W ramach projektu zaimplementowano w języku C++ listę dwukierunkową alokowaną na stercie, z wykorzystaniem wzorców Factory i Iterator. Wszystkie wymagane operacje (dodawanie i usuwanie na początku, końcu i pod wskazanym indeksem, wyświetlanie listy w obu kierunkach, ruch iteratora oraz czyszczenie) zostały zrealizowane i przetestowane w funkcji `main`, a wynik testu jest zgodny z oczekiwaniami.

Implementacja potwierdziła zalety listy dwukierunkowej: dodawanie i usuwanie na obu końcach działa w stałym czasie, a po liście można poruszać się w obu kierunkach. Ceną jest większe zużycie pamięci (dwa wskaźniki na element), brak bezpośredniego dostępu do elementu po indeksie (operacje pod indeks mają złożoność liniową) oraz konieczność ręcznego pilnowania wskaźników. Najwięcej uwagi wymagały przypadki brzegowe: operacje na pustej liście, liście jednoelementowej oraz aktualizacja wskaźników `head` i `tail`. Wzorzec Factory zebrał w jednym miejscu tworzenie węzłów, a wzorzec Iterator pozwolił przeglądać listę bez dostępu do jej wewnętrznej budowy.

Podczas analizy zauważono ograniczenia obecnej wersji, które można usunąć w dalszych pracach:

- metoda `removeAtIndex` nie sprawdza, czy indeks jest ujemny; dla ujemnego indeksu (innego niż zero) program odwołuje się do pustego wskaźnika,
- metody dodające i usuwające pod indeks bez komunikatu ignorują lub korygują indeks spoza zakresu; lepsza byłaby sygnalizacja błędu,
- iterator nie jest powiadamiany o usunięciu węzła, na który wskazuje, więc po usunięciu tego węzła z listy nie należy go używać,
- lista przechowuje tylko wartości typu `int`; uogólnienie można uzyskać przez zastosowanie szablonów (`template`),
- zamiast surowych wskaźników można zastosować inteligentne wskaźniki z biblioteki standardowej.

Praca z systemem Git i serwisem GitHub pozwoliła prześledzić historię zmian projektu, bezpiecznie cofać wprowadzone zmiany i odtwarzać utracone pliki. Pokazała też, że częste, małe commity z czytelnymi opisami znacznie ułatwiają wyszukiwanie błędów i powrót do poprawnej wersji. Narzędzie Doxygen pozwoliło zaś wygenerować dokumentację programu bezpośrednio z komentarzy w kodzie, dzięki czemu opis klas i metod jest zawsze zgodny z implementacją.

Literatura

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Wprowadzenie do algorytmów*, Wydawnictwo Naukowe PWN, Warszawa 2012.

[2] E. Gamma, R. Helm, R. Johnson, J. Vlissides, *Wzorce projektowe. Elementy oprogramowania obiektowego wielokrotnego użytku*, Helion, Gliwice 2010.

[3] B. Stroustrup, *Język C++. Kompendium wiedzy*, Helion, Gliwice 2014.

[4] *cppreference.com*, strona internetowa, https://en.cppreference.com.

[5] S. Chacon, B. Straub, *Pro Git*, wyd. 2, Apress 2014, https://git-scm.com/book.

[6] *GitHub Docs*, strona internetowa, https://docs.github.com.

[7] *Doxygen Manual*, strona internetowa, https://www.doxygen.nl/manual/index.html.

Spis rysunków

2.1. Struktura listy dwukierunkowej z trzema elementami 6

Spis tabel

1.1. Wymagania funkcjonalne 5
2.1. Złożoność operacji 7
3.1. Pliki projektu 9
3.2. Ustawienia Doxyfile 10

Spis listingów

1. Podstawowe polecenia Git 10
2. Przykład komentarza w formacie Doxygen 10
3. Klasa Node 12
4. Klasa NodeFactory 12
5. Klasa ListIterator 13
6. Metody addFront i addBack 13
7. Metoda addAtIndex 14
8. Metody removeFront, removeBack i clear 15
9. Metoda removeAtIndex 16
10. Funkcja main z testami 16
11. Wynik działania programu 17